

Recent Generated Responses: Feature Store Refresh, Partitioning, and Joins

Compiled from the most recent answers in this conversation

Purpose. This document copies the recent responses covering how to maintain a fixed observation window as new applications arrive, how to refresh each table, how to partition the data, what each partition means, and how to join origination features with performance features.

1. Maintaining a Fixed Observation Window as New Applications Arrive

Treat each application as its own cohort anchor.

Because the observation window is fixed, the right model is one origination row created when the application arrives, one performance row tied to that same application, and that performance row refreshed over time until the fixed observation window completes. The final join stays on the business key from origination.

You do not need a monthly full-history snapshot. You need a stateful rolling population of applications whose windows are still open.

1.1 Core design

Use two main tables and one final view:

- `application_snapshot_features`: one row per `application_id` + `applicant_id`, containing static origination features.
- `customer_application_12m_performance_features`: one row per `application_id` + `applicant_id`, containing `observation_start_date`, `observation_end_date`, `source_data_as_of_date`, `is_window_complete_flag`, `months_of_data_available`, and the 12-month aggregated performance features.
- `vw_monitoring_scorecard_customer`: a join of the two on `application_id` + `applicant_id`.

1.2 How to maintain it over time

On every run, process two groups:

- Brand new applications: applications that arrived since the last run. Insert them into `application_snapshot_features` and create an initial performance row in `customer_application_12m_performance_features`.
- Existing applications with open windows: applications already in the performance table where `is_window_complete_flag` = false. Recompute the aggregated performance features using all daily observations available so far and update the existing row.

The process each run is therefore:

- new applications + existing incomplete applications = applications to process this run

1.3 Driving refresh population

Build a driving set from both new rows and incomplete rows. Conceptually:

```
WITH apps_to_refresh AS (  
    SELECT a.application_id, a.applicant_id, a.customer_id, a.application_date  
    FROM application_snapshot_features a  
    LEFT JOIN customer_application_12m_performance_features p  
        ON a.application_id = p.application_id  
        AND a.applicant_id = p.applicant_id  
    WHERE p.application_id IS NULL  
  
    UNION  
  
    SELECT p.application_id, p.applicant_id, p.customer_id, p.observation_start_date AS  
application_date  
    FROM customer_application_12m_performance_features p  
    WHERE p.is_window_complete_flag = false  
)
```

1.4 Calculating the performance features

For each application/applicant in apps_to_refresh:

- observation_start_date = application_date
- observation_end_date = add_months(application_date, 12)
- join to daily performance rows inside that window and aggregate to the final performance row.

Recommended control columns in the performance table:

- observation_start_date
- observation_end_date
- source_data_as_of_date
- feature_calculated_at
- months_of_data_available
- is_window_complete_flag
- feature_status with values such as NEW, PARTIAL, COMPLETE

1.5 Lifecycle of one application

- First run after application arrives: create the origination row and a performance row with limited available data. is_window_complete_flag = false.
- Later runs: keep recomputing the same application/applicant performance row as more daily performance arrives.
- When the window is complete: recompute one last time, set is_window_complete_flag = true, and normally stop updating unless there is a correction or backfill.

1.6 Join with application-at-origin

Because both tables are at the same grain, the final join is straightforward:

```
SELECT a.*, p.observation_start_date, p.observation_end_date, p.source_data_as_of_date,  
        p.is_window_complete_flag, p.months_of_data_available,  
        p.max_dpd_12m, p.avg_balance_12m, p.payment_ratio_12m_avg  
FROM application_snapshot_features a
```

```
LEFT JOIN customer_application_12m_performance_features p
  ON a.application_id = p.application_id
  AND a.applicant_id = p.applicant_id;
```

2. Refreshing Each Table, Partitioning the Data, and How to Join

Refresh each table by population state, not by rerunning everything. Use three populations on every run: new applications not yet in feature tables, open-window applications already started but not yet complete, and closed-window applications whose windows are complete and normally do not change.

2.1 Table-by-table refresh model

application_base - Canonical application/applicant population, including included and excluded records.

- Refresh by inserting new applications and updating existing rows only if source application data or inclusion/exclusion flags changed.
- Partition by month(application_date).
- A partition means rows for applications whose business application date falls in that month, not rows loaded in that month.

application_snapshot_features - Origination-only features, one row per application/applicant.

- Refresh by inserting rows for new applications and updating only if origination logic is rerun or upstream application data is corrected.
- Partition by month(application_date).
- A partition means origination feature rows for applications made in that month.

eligible_application_contract_map - Links eligible applicants/customers to contracts.

- Refresh by inserting new mappings for new eligible applications and updating if contract mapping arrives late or changes.
- Partition by month(application_date) for a scorecard anchored to application cohorts.

contract_12m_performance_features - Contract-level performance features over the fixed observation window.

- Refresh by inserting rows for new applications/contracts entering the window, recomputing rows for open-window applications, and stopping refresh once is_window_complete_flag = true.
- Partition by month(observation_end_date).
- A partition means rows whose fixed observation window ends in that month.

customer_application_12m_performance_features - Rolled-up 12-month performance row at the monitoring grain.

- Refresh by inserting one row when a new application enters monitoring, updating the same row while the window is open, and freezing it once complete.
- Partition by month(observation_end_date).
- A partition means application/applicant monitoring rows whose 12-month window ends in that month.

vw_monitoring_scorecard_customer - Final joined monitoring scorecard view.

- No direct refresh if it remains a view. It reflects the current contents of the underlying tables.

2.2 Suggested partitioning plan

- application_base -> months(application_date)
- application_snapshot_features -> months(application_date)
- eligible_application_contract_map -> months(application_date)
- contract_12m_performance_features -> months(observation_end_date)
- customer_application_12m_performance_features -> months(observation_end_date)

Why this split: origination tables are queried by origination cohort, while fixed-window performance tables are commonly queried by window maturity/end date.

2.3 What each partition means

- months(application_date): rows for applications originated in that month. This is an origination cohort partition.
- months(observation_end_date): rows whose fixed observation window ends in that month. This is a maturity cohort partition.
- Partitions do not mean when the ETL ran, when data landed in storage, or when the row was last updated.

Operational timestamps such as pipeline_run_id, feature_calculated_at, source_data_as_of_date, created_at, and updated_at should be kept as metadata, not used as the main partitioning key.

2.4 Run order for the pipeline

- Refresh application_base with new and changed application data.
- Refresh application_snapshot_features from application_base.
- Refresh eligible_application_contract_map for the eligible population.
- Build apps_to_refresh from new applications plus incomplete performance rows.
- Recompute contract_12m_performance_features for that refresh population.
- Recompute customer_application_12m_performance_features by rolling up contract-level features.
- Upsert into the performance tables using MERGE.
- Expose the final result through vw_monitoring_scorecard_customer.

2.5 How to join

There are two important joins:

- Origination to performance: join application_snapshot_features to customer_application_12m_performance_features on application_id and applicant_id.
- Eligibility to contract observations: inside the refresh pipeline, join eligible application/applicant/customer records to contract mappings, then join to daily contract observations filtered to the fixed observation window.

Final monitoring scorecard join:

```
SELECT a.*, p.observation_start_date, p.observation_end_date, p.source_data_as_of_date,
       p.is_window_complete_flag, p.max_dpd_12m, p.avg_balance_12m, p.payment_ratio_12m_avg
FROM application_snapshot_features a
LEFT JOIN customer_application_12m_performance_features p
  ON a.application_id = p.application_id
 AND a.applicant_id   = p.applicant_id;
```

2.6 Freshness model

Separate three concepts:

- Business validity: what period the feature describes, represented by application_date, observation_start_date, and observation_end_date.
- Source freshness: how current the underlying source data is, represented by source_data_as_of_date.
- Processing freshness: when the feature was last computed, represented by feature_calculated_at and pipeline_run_id.

2.7 Short practical answer

- New applications -> insert into base and origination tables, initialize performance rows.
- Applications still in the observation window -> recompute and update performance rows.
- Applications whose windows are complete -> stop updating unless a correction or backfill is needed.
- Partition origination tables by application date month and fixed-window performance tables by observation end month.
- Join the final scorecard on application_id + applicant_id.

Monthly End Point-in-Time Row Design

Purpose. This section adds the recommended schema for storing monthly end point-in-time rows. This design is useful when you want one row per application and applicant at each month end, so monitoring can track how performance develops over time without storing cumulative full history in every monthly snapshot.

1. Recommended monthly point-in-time model

- Keep `application_snapshot_features` as the origination anchor table, one row per `application_id` + `applicant_id`.
- Create a monthly performance table with one row per `application_id` + `applicant_id` + `month_end_date`.
- Each monthly row should contain the state as of that month end only, not the whole prior history repeated inside the row.
- If contract-level observations exist, derive contract monthly features first and then roll them up to the application + applicant monthly grain.
- Use a separate final 12-month aggregate table only when you need a fixed-window outcome row for scorecard monitoring.

2. Table schema: `application_snapshot_features`

Purpose: application-at-origination feature table. This remains the anchor table that monthly point-in-time rows join back to.

Field group	Example fields
Keys and anchor	<code>application_id</code> , <code>applicant_id</code> , <code>customer_id</code> , <code>application_date</code> , <code>feature_timestamp</code>
Origination product features	<code>product_type</code> , <code>channel_code</code> , <code>requested_amount</code> , <code>approved_amount</code> , <code>term_months</code> , <code>interest_rate_at_origination</code> , <code>repayment_type</code> , <code>loan_purpose</code>
Applicant and affordability features	<code>age_at_application</code> , <code>employment_status_at_application</code> , <code>residential_status_at_application</code> , <code>declared_income_monthly</code> , <code>verified_income_monthly</code> , <code>debt_to_income_ratio_at_application</code> , <code>loan_to_income_ratio_at_application</code>
Bureau features	<code>bureau_score_at_application</code> , <code>active_credit_count_at_application</code> , <code>total_external_balance_at_application</code> , <code>ccj_count_at_application</code> , <code>default_count_at_application</code> , <code>worst_status_12m_at_application</code> , <code>worst_status_24m_at_application</code>
Score and decision features	<code>application_scorecard_score</code> , <code>application_score_band</code> , <code>origination_pd</code> , <code>decision_code</code>
Metadata	<code>feature_version</code> , <code>source_system</code> , <code>pipeline_run_id</code> , <code>created_at</code>

3. Table schema: contract_monthly_performance_features

Purpose: monthly contract-level point-in-time rows derived from daily contract observations. Create this when performance is naturally stored at contract or account level and later needs to be rolled up.

Field group	Example fields
Keys and time	application_id, applicant_id, customer_id, contract_id, month_end_date, feature_timestamp
Coverage / freshness	source_data_as_of_date, days_of_data_available_in_month, is_month_complete_flag
Month-end status	month_end_dpd, month_end_balance, month_end_limit, month_end_utilisation_ratio
Monthly delinquency	max_dpd_in_month, days_dpd_gt_0_in_month, days_dpd_ge_30_in_month, ever_30dpd_in_month_flag
Monthly payment features	payment_due_in_month, payment_made_in_month, payment_ratio_in_month, underpaid_in_month_flag, missed_payment_in_month_flag
To-date roll-forward features	max_dpd_to_date, months_in_arrears_to_date, max_consecutive_months_in_arrears_to_date
Status flags	forbearance_in_month_flag, restructure_in_month_flag, default_in_month_flag
Metadata	feature_version, source_system, pipeline_run_id, created_at

4. Table schema: customer_application_monthly_performance_features

Purpose: the main monthly point-in-time feature table at the monitoring grain. This is the table that should normally join back to application_snapshot_features.

Field group	Example fields
Keys and time	application_id, applicant_id, customer_id, month_end_date, feature_timestamp
Coverage / freshness	source_data_as_of_date, months_on_book, is_month_complete_flag
Monthly balance and exposure	month_end_balance, avg_balance_in_month, max_balance_in_month, avg_utilisation_ratio_in_month, max_utilisation_ratio_in_month
Monthly delinquency	month_end_dpd, max_dpd_in_month, months_with_dpd_gt_0_to_date, months_with_dpd_ge_30_to_date, ever_30dpd_to_date_flag
Monthly payment features	payment_due_in_month, payment_made_in_month, payment_ratio_in_month, months_underpaid_to_date, months_missed_payment_to_date

To-date monitoring features	max_dpd_to_date, avg_balance_to_date, max_consecutive_months_in_arrears_to_date, two_consecutive_months_dpd_gt_0_to_date_flag
Status flags	forbearance_to_date_flag, restructure_to_date_flag, default_to_date_flag
Metadata	feature_version, source_system, pipeline_run_id, created_at

5. Table schema: vw_monitoring_scorecard_customer_monthly

Purpose: monthly monitoring view that combines origination features with the month-end point-in-time performance row.

Field block	Example fields
Keys	application_id, applicant_id, customer_id, application_date, month_end_date
Origination features	requested_amount, term_months, declared_income_monthly, bureau_score_at_application, application_score_band, origination_pd
Monthly point-in-time features	months_on_book, month_end_dpd, max_dpd_in_month, month_end_balance, payment_ratio_in_month, max_dpd_to_date, months_with_dpd_ge_30_to_date, default_to_date_flag
Freshness fields	source_data_as_of_date, is_month_complete_flag, feature_timestamp

6. Partitioning and what each partition means

Object	Recommended partition	Meaning of one partition
application_snapshot_features	months(application_date)	One partition contains applications originated in that month
contract_monthly_performance_features	months(month_end_date)	One partition contains contract point-in-time rows for that month end
customer_application_monthly_performance_features	months(month_end_date)	One partition contains application + applicant monthly point-in-time rows for that month end

Important. Partition by business-effective date such as application_date or month_end_date, not by the ETL run date. The partition should describe the business time slice represented by the row, not when the pipeline happened to execute.

7. How the monthly join works

Join grain. The monthly point-in-time table should join back to the origination anchor on application_id + applicant_id. The monthly table carries month_end_date, while the origination table remains one row per application/applicant.

```
SELECT
    a.application_id,
    a.applicant_id,
    a.customer_id,
    a.application_date,
    a.requested_amount,
    a.bureau_score_at_application,
    m.month_end_date,
    m.months_on_book,
    m.month_end_dpd,
    m.max_dpd_in_month,
    m.month_end_balance,
    m.payment_ratio_in_month,
    m.max_dpd_to_date,
    m.months_with_dpd_ge_30_to_date,
    m.default_to_date_flag
FROM application_snapshot_features a
LEFT JOIN customer_application_monthly_performance_features m
    ON a.application_id = m.application_id
    AND a.applicant_id = m.applicant_id;
```

8. Refresh logic for monthly point-in-time rows

- When a new application arrives, insert one origination row into application_snapshot_features.
- At each month end, generate one new monthly point-in-time row for every application that has entered the observation population and is still active for monitoring.
- Only the current month-end partition normally gets newly written in the monthly point-in-time table; prior month partitions stay stable except for backfills or corrections.
- If late-arriving source data changes the month-end result, rebuild only the affected month_end_date partition and merge it back.
- Use a separate fixed-window table, such as customer_application_12m_performance_features, only when you need a final 12-month aggregate outcome in addition to the month-by-month view.

9. Recommended minimum set of tables for monthly point-in-time monitoring

Object	Purpose
application_snapshot_features	Application-at-origination anchor table
contract_monthly_performance_features	Optional contract-level monthly point-in-time rows when source observations are contract based
customer_application_monthly_performance_features	Main monthly monitoring feature table at application + applicant grain
vw_monitoring_scorecard_customer_monthly	Consumer-facing monthly monitoring view